Task 1: Execute the script shown in the video.

Terraform Basics — Input Variables and Variable Definition Methods

1. Objective

To understand Terraform input variables and implement five different methods of supplying values to Terraform configurations:

1. Using default values in variables.tf
2. Using interactive mode
3. Using command-line flags
4. Using environment variables
5. Using variable definition files, including automatically loaded .auto.tfvars files

2. Purpose

Input variables allow Terraform configurations to accept different values without modifying resource definitions. This improves reusability, maintainability, and flexibility across development, testing, and production environments.

In this practical, we use the local provider to create a text file and the random provider to generate a pet name.

3. Prerequisites

- Terraform installed on the local Windows machine.
- Vs Code terminal.
- A Terraform working directory named terraform_Basics.
- Required providers: hashicorp/local and hashicorp/random.

Verify the installation: terraform version
Initialize the project: terraform init

4. Terraform Resource Configuration

4.1 main.tf

```
resource "local_file" "my_pet" {
  filename = var.filename
  content  = var.content
}

resource "random_pet" "sunith" {
  prefix    = var.prefix
  separator = "."
  length    = 1
}
```

Explanation

- local_file.my_pet creates a text file on the local machine.
- filename determines the output file path.
- content specifies the text written into the file.
- random_pet.sunith generates a random pet name.

- prefix adds a prefix to the generated name.
- separator defines the separator between name components.
- length = 1 requests one random name component.

The resource configuration references the variables using var.filename, var.content, and var.prefix.

---

5. Question 1 — Using Default Values in variables.tf

Objective

To declare input variables and provide default values directly in the Terraform configuration.

Step 1: Create variables.tf

```
variable "filename" {
  default = "sunitChintu2.txt"
}

variable "content" {
  default = "My Aim is to only move Forward2"
}

variable "prefix" {
  default = "malem"
}
```

Step 2: Validate the configuration

```
terraform validate
```

Expected output: Success! The configuration is valid.

Step 3: Preview and apply

```
→ terraform plan
→ terraform apply
```

Type yes when prompted.

Step 4: Verify the result

cat sunitChintu2.txt

Expected output: My Aim is to only move Forward2

Key learning

When a variable has a default value, Terraform uses that value if no higher-precedence value is supplied.

---

6. Question 2 — Interactive Mode

Objective

To provide variable values interactively when Terraform prompts for variables that have no default values and have not been supplied through another mechanism.

Step 1: Declare variables without defaults

Update variables.tf:

```
variable "filename" {

 }
variable "content" {

}
variable "prefix" {

}
```

Step 2: Run Terraform apply

```
terraform apply
```

Because the variables have no defaults and no other values have been supplied, Terraform prompts for them.

Example prompts:

var.content

 Enter a value: My Aim is to only move Forward2

var.filename

 Enter a value: sunitChintu2.txt

var.prefix

 Enter a value: malem

Terraform uses the supplied values to create or update the resources.

Important note

Interactive prompting occurs only when Terraform needs a value and cannot obtain one from another source. Variables declared as optional through defaults, or supplied through other mechanisms, do not need interactive input.

Key learning

Interactive mode is useful for quick experiments, but it is generally less suitable for automated CI/CD pipelines because it requires user input.

---

7. Question 3 — Command-Line Flags

Objective

To supply input variable values directly through Terraform CLI arguments.

Example command

```
terraform apply -var="filename=pets.txt" -var="prefix=vennela" -var="content=My pet name is Snupi"
```

Explanation

- -var="filename=pets.txt" sets the filename variable.
- -var="prefix=vennela" sets the prefix variable.
- -var="content=My pet name is Snupi" sets the content variable.

Terraform uses these supplied values instead of the defaults or lower-precedence values for the same variables.

The /pets.txt path is an example of a Linux path. When running Terraform on Windows, choose an appropriate Windows path or a path supported by your environment.

Key learning

The -var option is useful when supplying a small number of values for one-off commands. Be careful not to expose sensitive values in shell history or process arguments.

---

8. Question 4 — Environment Variables

Objective

To provide Terraform input values through environment variables rather than entering them directly into the Terraform command.

Step 1: Declare variables

Use the following variables.tf:

```
variable "filename" {

}
variable "content" {

}
variable "prefix" {

}
```

Step 2: Export environment variables
In vs Code:
→ export TF_VAR_filename="sunith"
→ export TF_VAR_content="my pet name is snupi"
→ export TF_VAR_prefix="vennela"

Explanation

Terraform automatically recognizes environment variables that begin with TF_VAR_.

Environment variable	Terraform input variable	Value
TF_VAR_filename	filename	sunith
TF_VAR_content	content	my pet name is snupi
TF_VAR_prefix	prefix	vennela

Step 3: Apply the configuration

→ terraform validate
→ terraform plan
→ terraform apply

Because the required values are available in the environment, Terraform does not need to prompt for them.

Step 4: Verify the results

cat sunith
→terraform state show random_pet.sunith
Expected file content: my pet name is snupi
The generated pet name uses the vennela prefix.

Important note

These exported variables remain available to child processes launched from the current Git Bash session. They are not automatically permanent system-wide settings.

To remove them from the current session:

→ unset TF_VAR_filename
→ unset TF_VAR_content
→ unset TF_VAR_prefix

Key learning

Environment variables are useful in automation and CI/CD systems, but sensitive values must be managed carefully.

---

9. Question 5 — Variable Definition Files

Objective

To understand how Terraform loads variable values from **.tfvars** and **.auto.tfvars files.**

Purpose

Variable definition files separate input values from resource configuration. This makes it easier to maintain different values for different environments without editing main.tf.

9.1 Declare variables in variables.tf

The final declaration file used in this practical is:

```
variable "filename" {

}
variable "content" {

}
variable "prefix" {

}
```

This file declares the variables but does not assign their values.

9.2 Method A — Custom variable definition file

Create a file named **variables.tfvars**

```
filename = "sunitChintu2.txt"
content  = "My Aim is to only move Forward2"
prefix   = "malem"
```

Unlike terraform.tfvars, a custom filename such as variables.tfvars is not automatically loaded.

Run:

→ terraform validate
→ terraform plan -var-file variables.tfvars
→ terraform apply -var-file="variables.tfvars"
Terraform reads the supplied values and uses them to configure the resources.

9.3 Error encountered and resolution

Error: Variable declaration in .tfvars file

Initially, variable declaration blocks were placed inside variables.tfvars. Terraform reported an error because .tfvars files assign values to variables rather than declare them.

Resolution:

- Declare variables in variables.tf.
- Assign values in variables.tfvars.
- Pass the custom file using -var-file.

Error: Reference to undeclared input variable

Terraform could not find the declarations for filename, content, and prefix.

Resolution:

The variable declaration file was recreated with the three required empty variable blocks. Terraform could then recognize the variables and obtain their values from the specified file.

9.4 Method B — Automatically loaded terraform.tfvars

Terraform automatically loads a file named terraform.tfvars or terraform.tfvars.json from the working directory.

Example terraform.tfvars:

```
filename = "sunitChintu2.txt"
content  = "My Aim is to only move Forward2"
prefix   = "malem"
```

Run:

```
→ terraform plan
→ terraform apply
```

No -var-file argument is required.

9.5 Method C — Automatically loaded .auto.tfvars

This is the final method successfully tested in this practical.

Create a file named variables.auto.tfvars

```
filename = "sunitChintu23.txt"
content  = "Testing automatic variable loading with auto.tfvars"
prefix   = "malem"
```

The * in the pattern *.auto.tfvars represents any matching filename prefix. It is a wildcard pattern, not a literal character that must be typed into the filename.

Examples of valid filenames include:

- variables.auto.tfvars
- dev.auto.tfvars
- production.auto.tfvars
- sunith.auto.tfvars

All must be located in the Terraform working directory to be automatically loaded.

9.6 Method D — JSON variable definition files

Terraform also supports JSON variable definition files.

Example filename: dev.auto.tfvars.json

```
{
  "filename": "sunitChintu23.txt",
  "content": "Testing automatic variable loading with JSON",
  "prefix": "malem"
}
```

Terraform automatically loads this file because it matches the *.auto.tfvars.json naming pattern.
For a custom JSON filename that does not match the automatic loading patterns, specify it explicitly:

```
→ terraform plan -var-file="dev.tfvars.json"
```

→ terraform apply -var-file="dev.tfvars.json"

9.7 Important distinction between filenames

Filename	Automatically loaded?
terraform.tfvars	Yes
terraform.tfvars.json	Yes
variables.auto.tfvars	Yes
dev.auto.tfvars	Yes
dev.auto.tfvars.json	Yes
variables.tfvars	No
dev.tfvars.json	No
variables.tf	Loaded as Terraform configuration, not as a variable definition file

9.8 Variable precedence

When the same variable is assigned multiple values, Terraform follows a precedence order. In simplified order from lower to higher precedence:

1. Variable defaults declared in .tf files.
2. Automatically loaded variable files such as terraform.tfvars and *.auto.tfvars, with later-loaded files able to override earlier ones.
3. Environment variables named TF_VAR_name.
4. Variable definition files explicitly supplied using -var-file.
5. Explicit -var command-line arguments.

When multiple -var-file arguments are supplied, later arguments take precedence over earlier ones. Exact file ordering matters when multiple automatically loaded files define the same variable.

For this practical, only one variable definition file was used at a time, so there was no ambiguity.

Key learning

- variables.tf declares variables.
- variables.tfvars supplies values when explicitly passed with -var-file.
- terraform.tfvars and terraform.tfvars.json are automatically loaded.
- *.auto.tfvars and *.auto.tfvars.json are automatically loaded.
- The * is a wildcard representing the filename prefix, not part of the actual filename.

9.9: Generate a Random Pet Name and Write It to a File

1. Terraform Configuration — main.tf

```
resource "local_file" "my_pet" {
  filename = "sunith2.txt"
  content  = "this is random generated ${random_pet.sunith.id}"
}

resource "random_pet" "sunith" {
  prefix    = "malem"
  separator = "."
  length    = 1
}
```

5. Execution Steps and Commands

Step 1: Initialize Terraform

terraform init

Initializes the working directory and installs the required providers.

Step 2: Format the configuration

terraform fmt

Formats the Terraform configuration.

Step 3: Validate the configuration

terraform validate

Expected result: Success! The configuration is valid.

Step 4: Preview the execution plan

terraform plan
Review the resources Terraform plans to create.

Step 5: Apply the configuration

terraform apply
Type yes when prompted.

6. Understanding the Resource Dependency

The expression: ${random_pet.sunith.id}

creates an **implicit dependency** between local_file.my_pet and random_pet.sunith.

Terraform understands that it must obtain the generated pet ID before it can determine the file content. Therefore, it creates the random pet first and then creates the file.

No explicit depends_on declaration is required.

7. Conclusion

This practical demonstrates Terraform string interpolation, resource attribute references, and implicit dependencies. A generated value from one resource is dynamically reused by another resource, eliminating the need to hardcode the generated name.

10: Display the Generated Pet Name Using an Output Block

3. Terraform Configuration — main.tf

```
resource "local_file" "my_pet" {
  filename = "sunith2.txt"
  content  = "this is random generated ${random_pet.sunith.id}"
}

resource "random_pet" "sunith" {
  prefix    = "malem"
  separator = "."
  length    = 1
}

output "my_pet" {
  value = "random_pet.sunith.id"
}
```

4. Important Correction to the Output Block

Alternatively, you can write:

```
output "my_pet" {

  value = "${random_pet.sunith.id}"

}
```

For this simple example, the direct reference without interpolation syntax is preferred.

5. Execution Steps and Commands

Step 1: apply

```
terraform apply
```

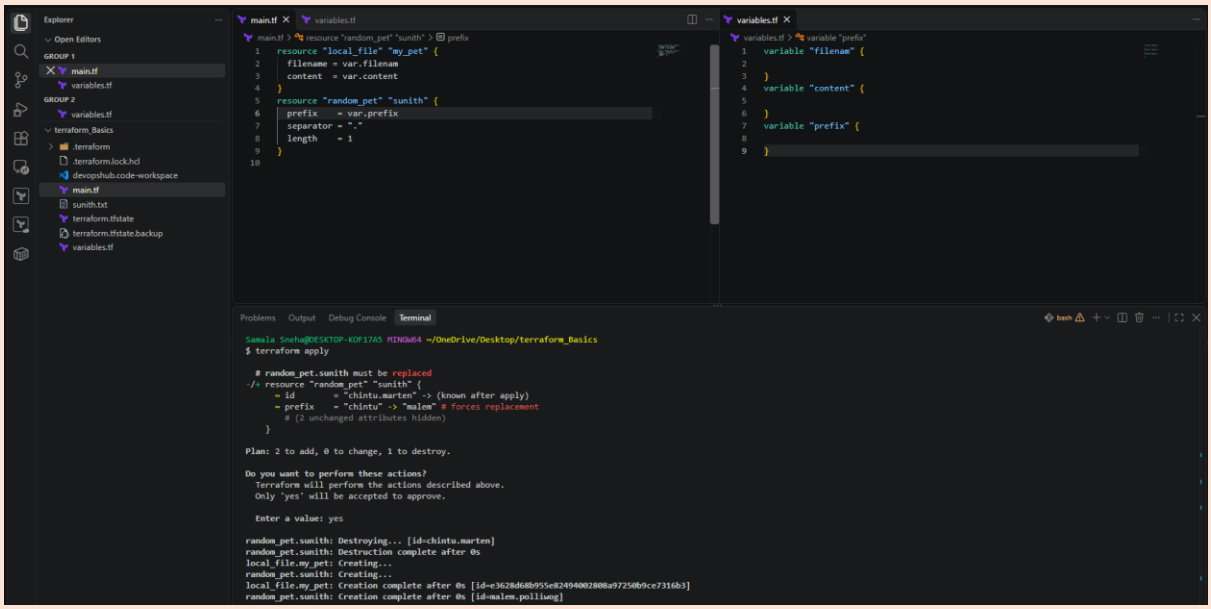
Step 5: Retrieve the output again

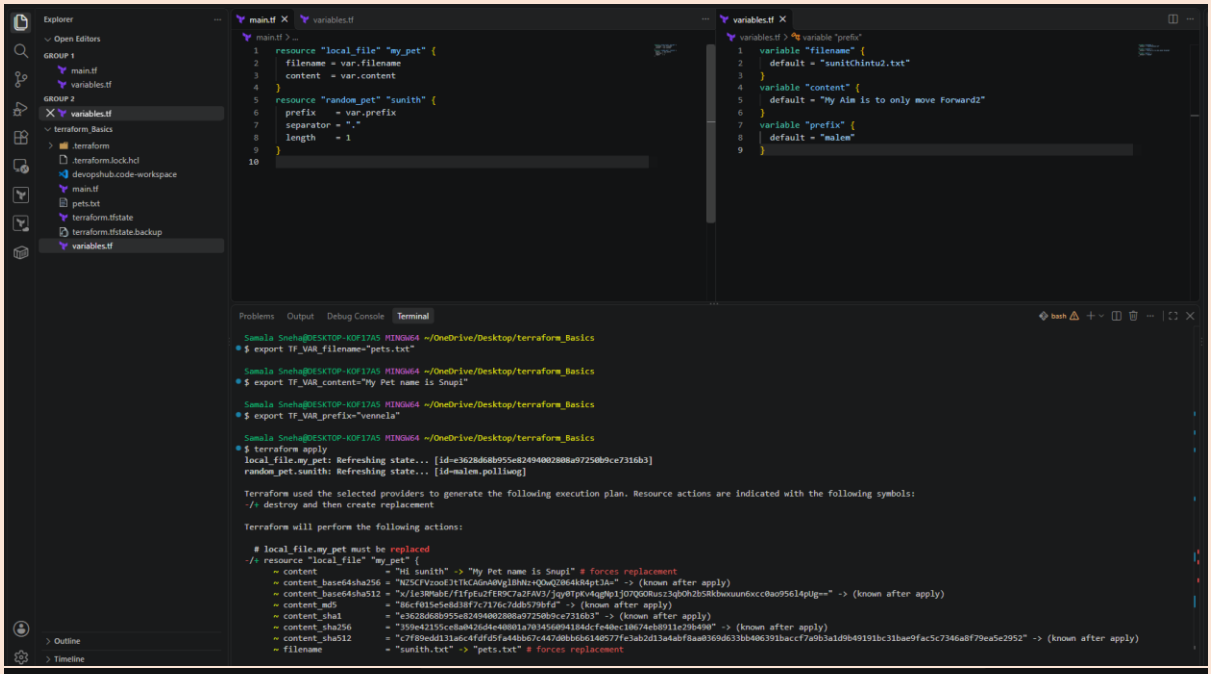
```
terraform output
my_pet = "malem.grizzly"
```

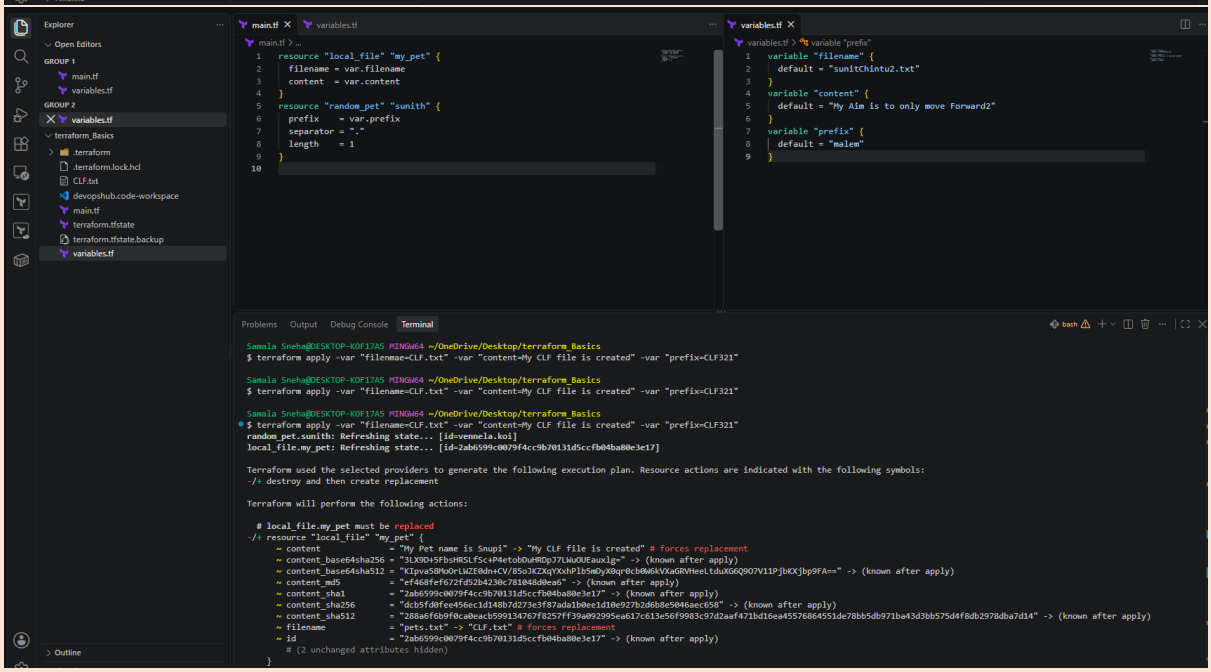
10. Summary of All Five Methods

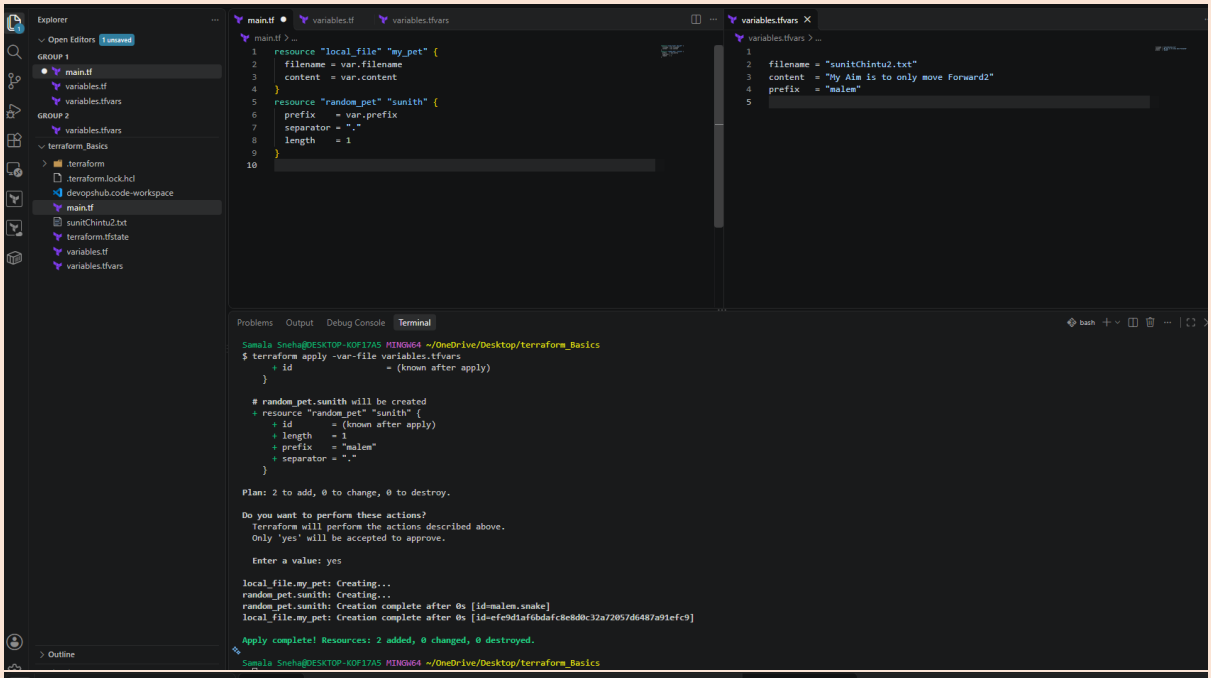
Method	Where values are provided	Typical command
Default values	variables.tf	terraform apply
Interactive mode	Terminal prompts	terraform apply
Command-line flags	-var arguments	terraform apply -var="prefix=MR"
Environment variables	TF_VAR_ variables	terraform apply
Variable definition files	.tfvars / .auto.tfvars files	terraform apply -var-file="variables.tfvars" or terraform apply for automatically loaded files

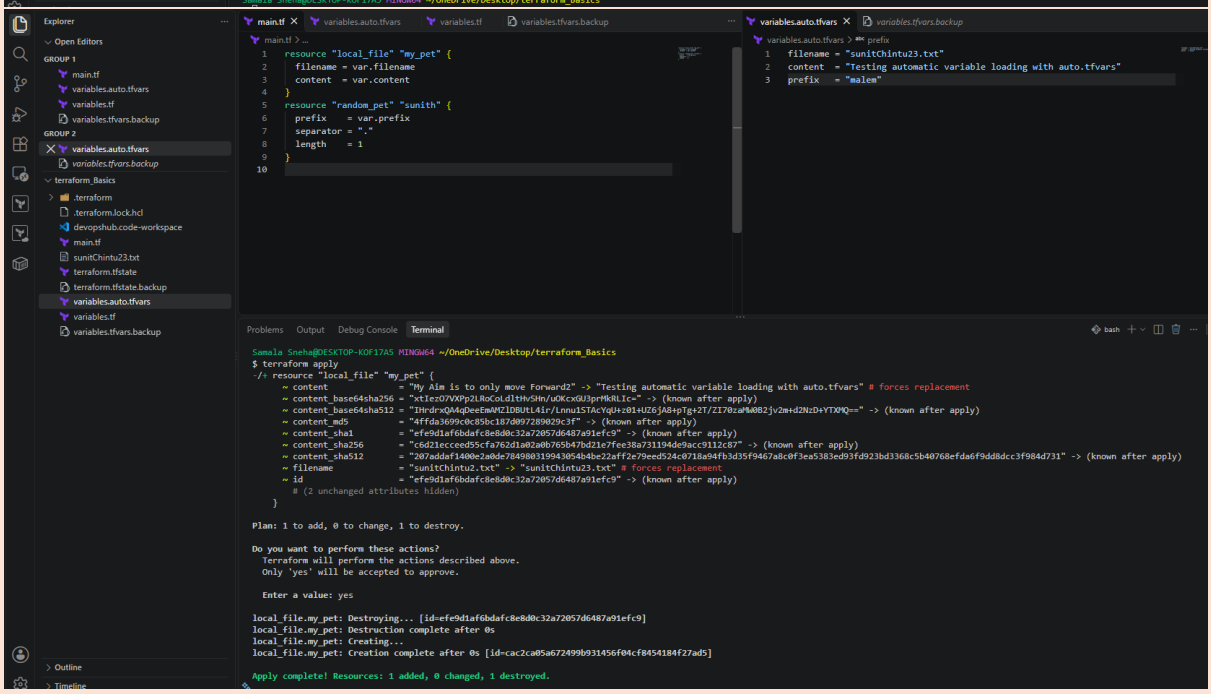
11. Implementation

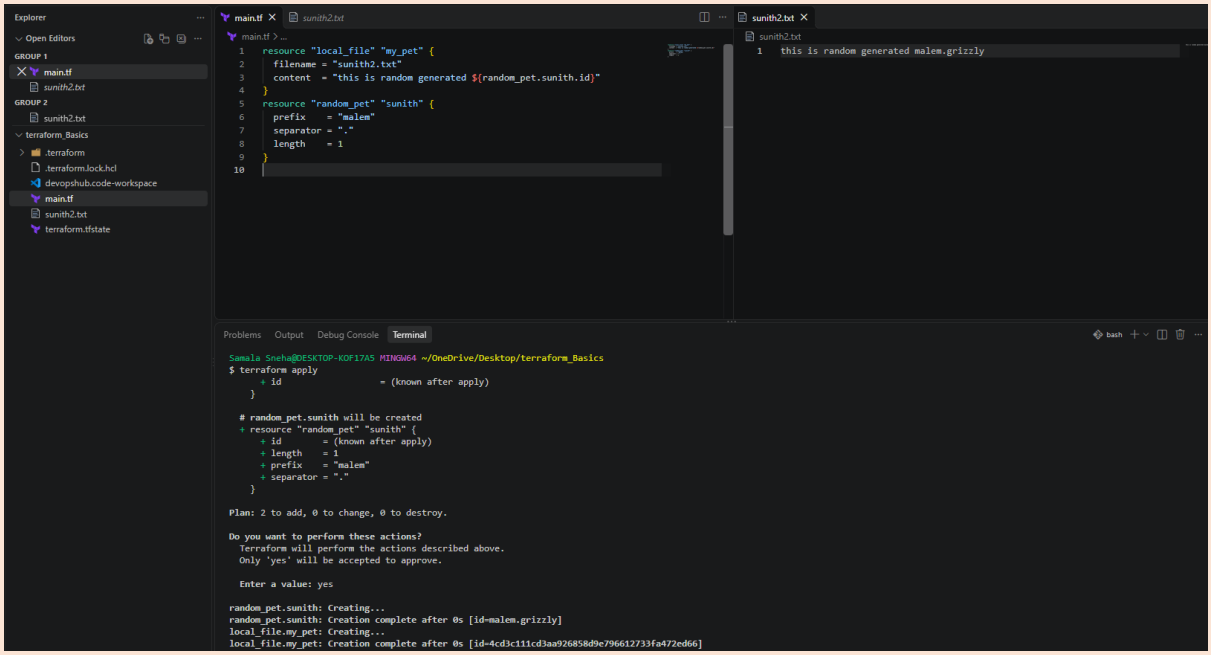


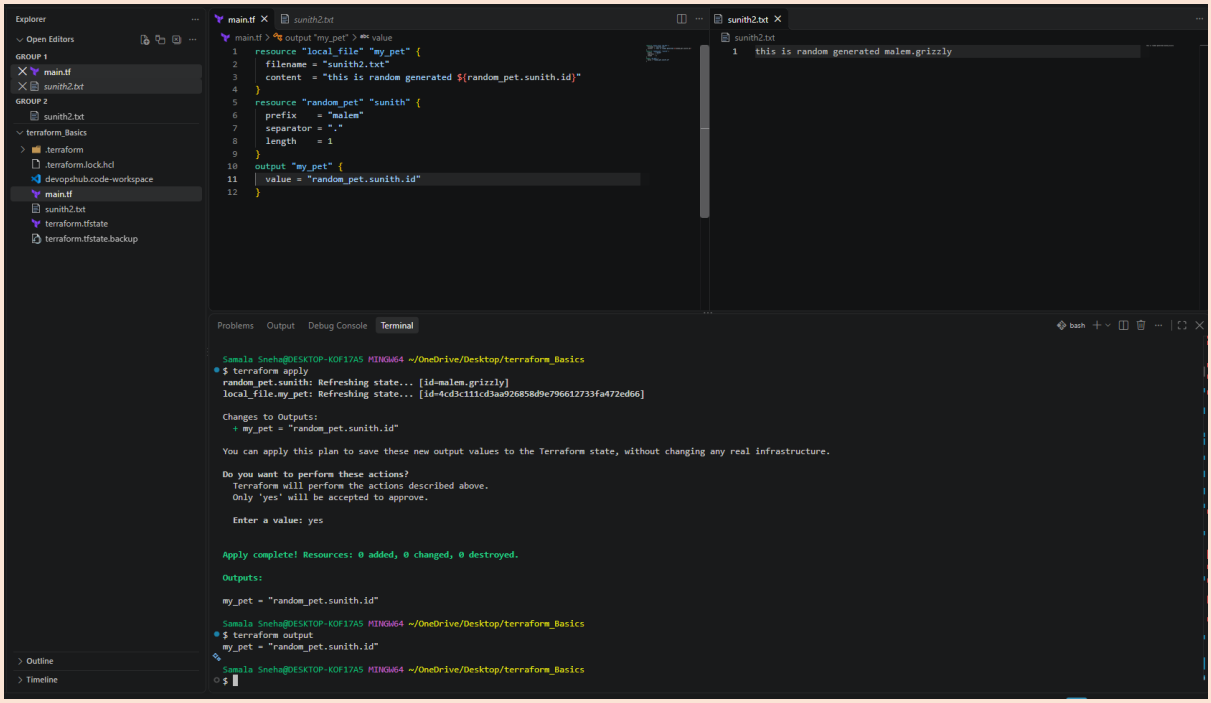

Explorer
Open Editors
GROUP 1
main.tf
variables.tf
GROUP 2
variables.tf
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
variables.tf
Outline
Timeline
main.tf
variables.tf
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "sunith" {
prefix = var.prefix
separator = "."
length = 1
}
variable "filename" {
default = "sunitChintu2.txt"
}
variable "content" {
default = "My Aim is to only move Forward2"
}
variable "prefix" {
default = "malem"
}
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ export TF_VAR_filename="pets.txt"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ export TF_VAR_content="My Pet name is Snupi"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ export TF_VAR_prefix="vennela"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
local_file.my_pet: Refreshing state... [id=e3628d68b955e8249402808a97250b9ce7316b3]
random_pet.sunith: Refreshing state... [id=malem.polliwog]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
~ content = "Hi sunith" -> "My Pet name is Snupi" # forces replacement
~ content_base64sha256 = "NZ5CFVzooE3tTkCAGnAVglBhNtrQOwQ2064k84pt3Ac=" -> (known after apply)
~ content_base64sha512 = "v/1e3RPabE/fifpEu2fER9C7a2FN3/3qv0TpKv4ugNp1JQ7QGORusz3gbCh2b58kbwxuun6xcc0ao956l4pUg==" -> (known after apply)
~ content_md5 = "86cf015e5e8d38f7c7176c7ddb579bfd" -> (known after apply)
~ content_sha1 = "e3628d68b955e8249402808a97250b9ce7316b3" -> (known after apply)
~ content_sha256 = "359e42155ce8a0426d4e40801a70345604184dcfe40ec10674eb8911e29b490" -> (known after apply)
~ content_sha512 = "c7f89edd131a6c4fdfd5fa44bb67c447d0bb6b6140577fe3ab2d13a4abf8aa0369d633bb406391baccf7a9b3a1d9b49191bc31bae9fac5c7346a8f79ea5e2952" -> (known after apply)
~ filename = "sunith.txt" -> "pets.txt" # forces replacement

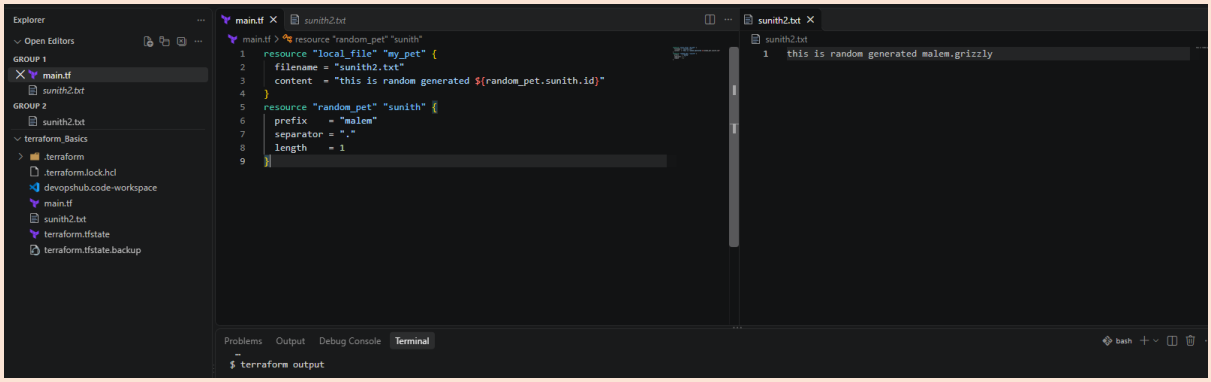

Explorer
Open Editors
GROUP 1
main.tf
variables.tf
GROUP 2
variables.tf
terraform_Basics
.terraform
.terraform.lock.hcl
CLF.txt
devopshub.code-workspace
main.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf
Outline
main.tf
variables.tf
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "sunith" {
prefix = var.prefix
separator = "."
length = 1
}
variable "filename" {
default = "sunitChintu2.txt"
}
variable "content" {
default = "My Aim is to only move Forward2"
}
variable "prefix" {
default = "malem"
}
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply -var "filenmae=CLF.txt" -var "content=My CLF file is created" -var "prefix=CLF321"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply -var "filename=CLF.txt" -var "content=My CLF file is created" -var "prefix=CLF321"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply -var "filename=CLF.txt" -var "content=My CLF file is created" -var "prefix=CLF321"
random_pet.sunith: Refreshing state... [id=vennela.koi]
local_file.my_pet: Refreshing state... [id=2ab6599c0079f4cc9b70131d5ccfb04ba80e3e17]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
~ content = "My Pet name is Snupi" -> "My CLF file is created" # forces replacement
~ content_base64sha256 = "3LX9D+5FbsHRSLfSc+P4etobDuHRDp37LkwOUEauxlg=" -> (known after apply)
~ content_base64sha512 = "KIpva58MoOrLNZE0dn+CV/85oJKZXqYXxhPlb5mDyX0qr0cb0W6kVXaGRVHeeLtduXG6Q9O7V11PjbKXjbp9FA==" -> (known after apply)
~ content_md5 = "ef468fef672fd52b4230c781048d0ea6" -> (known after apply)
~ content_sha1 = "2ab6599c0079f4cc9b70131d5ccfb04ba80e3e17" -> (known after apply)
~ content_sha256 = "dcb5fd0fee456ec1d148b7d273e3f87ada1b0ee1d10e927b2d6b8e5046aec658" -> (known after apply)
~ content_sha512 = "288a6f6b9f0ca0eacb599134767f8257ff39a092995ea617c613e56f9983c97d2aaf471bd16ea4557686455 1de78bb5db971ba43d3bb575d4f8db2978dba7d14" -> (known after apply)
~ filename = "pets.txt" -> "CLF.txt" # forces replacement
~ id = "2ab6599c0079f4cc9b70131d5ccfb04ba80e3e17" -> (known after apply)
(2 unchanged attributes hidden)
}


Explorer
Open Editors 1 unsaved
GROUP 1
main.tf
variables.tf
variables.tfvars
GROUP 2
variables.tfvars
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
sunitChintu2.txt
terraform.tfstate
variables.tf
variables.tfvars
Outline
main.tf
variables.tf
variables.tfvars
main.tf > ...
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "sunith" {
prefix = var.prefix
separator = "."
length = 1
}
variables.tfvars
variables.tfvars > ...
filename = "sunitChintu2.txt"
content = "My Aim is to only move Forward2"
prefix = "malem"
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply -var-file variables.tfvars
+ id = (known after apply)
}
random_pet.sunith will be created
+ resource "random_pet" "sunith" {
+ id = (known after apply)
+ length = 1
+ prefix = "malem"
+ separator = "."
}
Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my_pet: Creating...
random_pet.sunith: Creating...
random_pet.sunith: Creation complete after 0s [id=malem.snake]
local_file.my_pet: Creation complete after 0s [id=efe9d1af6bdafc8e8d0c32a72057d6487a91efc9]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics


Explorer
Open Editors
GROUP 1
main.tf
variables.auto.tfvars
variables.tf
variables.tfvars.backup
GROUP 2
variables.auto.tfvars
variables.tfvars.backup
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
sunitChintu23.txt
terraform.tfstate
terraform.tfstate.backup
variables.auto.tfvars
variables.tf
variables.tfvars.backup
Outline
Timeline
main.tf
variables.auto.tfvars
variables.tf
variables.tfvars.backup
main.tf > ...
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "sunith" {
prefix = var.prefix
separator = "."
length = 1
}
variables.auto.tfvars
variables.tfvars.backup
variables.auto.tfvars > prefix
filename = "sunitChintu23.txt"
content = "Testing automatic variable loading with auto.tfvars"
prefix = "malem"
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
-/+ resource "local_file" "my_pet" {
~ content = "My Aim is to only move Forward2" -> "Testing automatic variable loading with auto.tfvars" # forces replacement
~ content_base64sha256 = "xtIezO7VXPp2LRoCoLdltHvSHn/uOKcxGU3prMkRLIc=" -> (known after apply)
~ content_base64sha512 = "1HrdrxQA4qDeeEmMZlDBUtL4ir/Lnnu1STAcYqU+z01+UZ6jA8+pTg+2T/ZI70zaM%0B2jv2m+d2NzD+YTXMQ==" -> (known after apply)
~ content_md5 = "4ffda3609c0c85bc187d097289029c3f" -> (known after apply)
~ content_sha1 = "efe9d1af6bdafc8e8d0c32a72057d6487a91efc9" -> (known after apply)
~ content_sha256 = "c6d21eccced55cfa762d1a02a0b765b47bd21e7fee38a731194de9acc9112c87" -> (known after apply)
~ content_sha512 = "207addaf1400e2a0de784980319943054b4be22aff2e79eed524c0718a94fb3d35f9467a8c0f3ea5383ed93fd923bd3368c5b40768efda6f9dd8dcc3f984d731" -> (known after apply)
~ filename = "sunitChintu2.txt" -> "sunitChintu23.txt" # forces replacement
~ id = "efe9d1af6bdafc8e8d0c32a72057d6487a91efc9" -> (known after apply)
(2 unchanged attributes hidden)
}
Plan: 1 to add, 0 to change, 1 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
local_file.my_pet: Destroying... [id=efe9d1af6bdafc8e8d0c32a72057d6487a91efc9]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=cac2ca05a672499b931456f04cf8454184f27ad5]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.


Explorer
Open Editors
GROUP 1
main.tf
sunith2.txt
GROUP 2
sunith2.txt
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
sunith2.txt
terraform.tfstate
main.tf
sunith2.txt
resource "local_file" "my_pet" {
filename = "sunith2.txt"
content = "this is random generated ${random_pet.sunith.id}"
}
resource "random_pet" "sunith" {
prefix = "malem"
separator = "."
length = 1
}
sunith2.txt
this is random generated malem.grizzly
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
+ id = (known after apply)
}
random_pet.sunith will be created
+ resource "random_pet" "sunith" {
+ id = (known after apply)
+ length = 1
+ prefix = "malem"
+ separator = "."
}
Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.sunith: Creating...
random_pet.sunith: Creation complete after 0s [id=malem.grizzly]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=4cd3c111cd3aa926858d9e796612733fa472ed66]


Explorer
Open Editors
GROUP 1
main.tf
sunith2.txt
GROUP 2
sunith2.txt
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
sunith2.txt
terraform.tfstate
terraform.tfstate.backup
Outline
Timeline
main.tf
sunith2.txt
resource "local_file" "my_pet" {
filename = "sunith2.txt"
content = "this is random generated ${random_pet.sunith.id}"
}
resource "random_pet" "sunith" {
prefix = "malem"
separator = "."
length = 1
}
output "my_pet" {
value = "random_pet.sunith.id"
}
sunith2.txt
this is random generated malem.grizzly
Problems
Output
Debug Console
Terminal
bash
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform apply
random_pet.sunith: Refreshing state... [id=malem.grizzly]
local_file.my_pet: Refreshing state... [id=4cd3c111cd3aa926858d9e796612733fa472ed66]
Changes to Outputs:
+ my_pet = "random_pet.sunith.id"
You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
Outputs:
my_pet = "random_pet.sunith.id"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$ terraform output
my_pet = "random_pet.sunith.id"
Samala Sneha@DESKTOP-KOF17A5 MINGW64 ~/OneDrive/Desktop/terraform_Basics
$


Explorer
Open Editors
GROUP 1
main.tf
sunith2.txt
GROUP 2
sunith2.txt
terraform_Basics
.terraform
.terraform.lock.hcl
devopshub.code-workspace
main.tf
sunith2.txt
terraform.tfstate
terraform.tfstate.backup
main.tf
sunith2.txt
resource "local_file" "my_pet" {
filename = "sunith2.txt"
content = "this is random generated ${random_pet.sunith.id}"
}
resource "random_pet" "sunith" {
prefix = "malem"
separator = "."
length = 1
}
sunith2.txt
this is random generated malem.grizzly
Problems
Output
Debug Console
Terminal
bash
$ terraform output

12. Conclusion

This practical demonstrated five ways to supply input variables to Terraform: default values, interactive prompts, command-line flags, environment variables, and variable definition files.

The work also demonstrated the difference between declaring variables in variables.tf, supplying values through a custom .tfvars file, and automatically loading values through .auto.tfvars.

The final .auto.tfvars test was completed successfully. Terraform recognized the automatically loaded values without requiring the -var-file option, and the local file was planned for replacement and applied with the new filename and content.

These concepts provide a foundation for building reusable Terraform configurations and managing environment-specific settings in DevOps workflows.

Task 2: Integrate Terraform in Jenkins Using Terraform Plugin

1. Objective

To integrate Terraform with Jenkins using a Declarative Pipeline and automate the creation of resources using Terraform configuration.

2. Purpose

The purpose of this task is to understand how Jenkins Pipeline can execute Terraform commands to initialize providers, create an execution plan, apply the configuration, and verify the generated resources.

In this lab, Terraform creates a local text file and generates a random pet name using the local and random providers. No AWS infrastructure is created.

3. Implemented Clear Steps

Step 1: Install and Verify Terraform

Terraform was installed on the Amazon EC2 instance.

Command: terraform -version

Output:
Terraform v1.17.0
on linux_amd64

Terraform executable path: /usr/bin/terraform

The Jenkins service user was also verified to have permission to execute Terraform:
sudo -u jenkins /usr/bin/terraform -version

Step 2: Configure Terraform in Jenkins

1. Log in to the Jenkins dashboard.
2. Navigate to **Manage Jenkins → Tools**.
3. Locate **Terraform installations**.
4. Configure the following values:
    - Name: Terraform-Local

- Install directory: /usr/bin
- Install automatically: Unchecked

5. Click **Save**.

This configuration identifies the existing Terraform installation for Jenkins.

Step 3: Create a Jenkins Pipeline Job

1. Open the Jenkins Dashboard.
2. Click **New Item**.
3. Enter the job name: terraform-pipeline-lab
4. Select **Pipeline**.
5. Click **OK**.
6. Scroll to the **Pipeline** section.
7. Set **Definition** to Pipeline script.
8. Paste the following Declarative Pipeline script.

Step 4: Configure the Pipeline Script

```
pipeline {
   agent any

   stages {
      stage('Create Terraform Files') {
         steps {
            writeFile file: 'main.tf', text: '''
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
    }
    random = {
      source = "hashicorp/random"
    }
  }
}

resource "local_file" "my_pet" {
  filename = "sunitChintu2.txt"
  content  = "My Aim is to only move Forward2"
}

resource "random_pet" "sunith" {
  prefix    = "chintu"
  separator = "."
  length    = 1
}
'''
         }
      }
```

```
        stage('Terraform Init') {
            steps {
                sh '/usr/bin/terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh '/usr/bin/terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh '/usr/bin/terraform apply -auto-approve'
            }
        }

        stage('Verify Output') {
            steps {
                sh '''
                    cat sunitChintu2.txt
                    /usr/bin/terraform output || true
                    ls -l sunitChintu2.txt
                '''
            }
        }
    }
}
```

Click **Save**.

Step 5: Execute the Pipeline

1. Open the terraform-pipeline-lab job.
2. Click **Build Now**.
3. Click the generated build number.
4. Open **Console Output**.
5. Verify that the stages execute successfully and the build finishes with:

Finished: SUCCESS

The Pipeline performs these operations:

- **Create Terraform Files:** Creates main.tf inside the Jenkins workspace.
- **Terraform Init:** Downloads and initializes the required providers.
- **Terraform Plan:** Checks the changes Terraform intends to make.
- **Terraform Apply:** Creates the resources described in the configuration.
- **Verify Output:** Displays the generated text file and checks that it exists.

Step 6: Verify the Generated File

The Jenkins workspace is: /var/lib/jenkins/workspace/terraform-pipeline-lab/

Run the following command on the EC2 instance:
sudo cat /var/lib/jenkins/workspace/terraform-pipeline-lab/sunitChintu2.txt

Expected output: My Aim is to only move Forward2

The file was successfully created, and its contents matched the Terraform configuration.

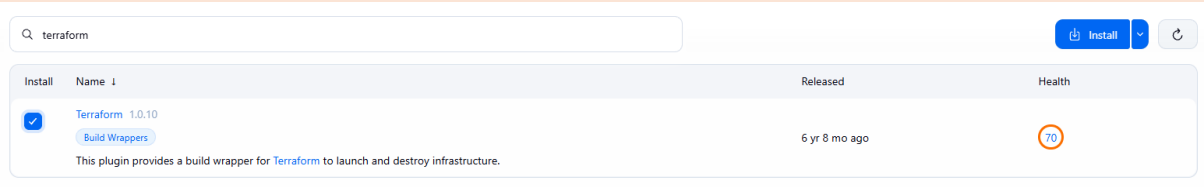
The workspace also contains:

- main.tf — Terraform configuration file.
- sunitChintu2.txt — File created by the local_file resource.
- terraform.tfstate — Terraform state file tracking the managed resources.

The random_pet.sunith resource generates a name with the chintu prefix and a period separator. Its generated name can be checked in the Terraform state or with an appropriate Terraform output.

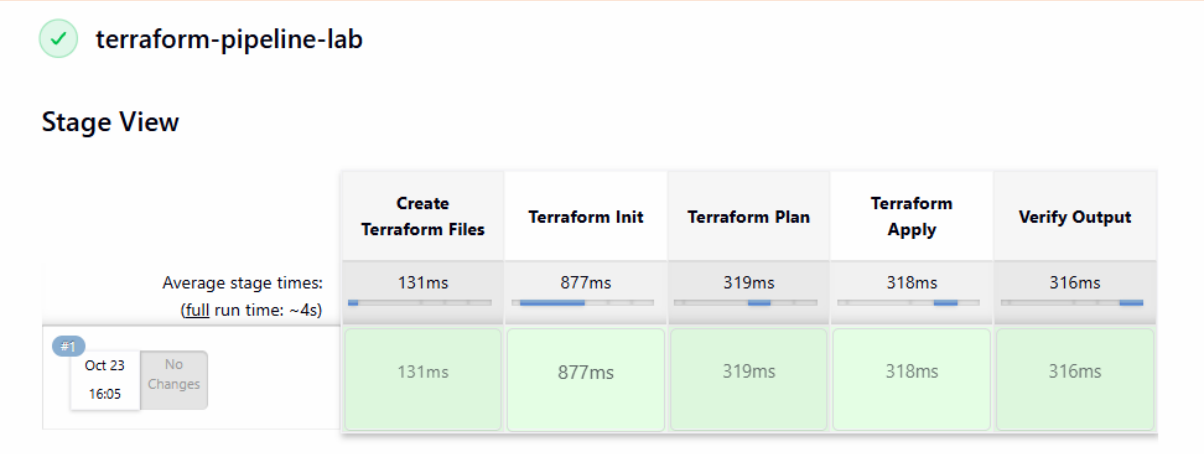
Important: The terraform.tfstate file contains resource state information. Do not publish it to a public repository.

Implementation:




```
[1mPlan:[0m [0m2 to add, 0 to change, 0 to destroy.
[0m[1mrandom_pet.sunith: Creating...[0m[0m
[0m[1mlocal_file.my_pet: Creating...[0m[0m
[0m[1mlocal_file.my_pet: Creation complete after 0s [id=efe9d1af6bdafc8e8d0c32a72057d6487a91efc9][0m
[0m[1mrandom_pet.sunith: Creation complete after 0s [id=chintu.asp][0m
[0m[1m[32m
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Verify Output)
[Pipeline] sh
+ cat sunitChintu2.txt
My Aim is to only move Forward2+ /usr/bin/terraform output
[33m[0m[0m
[33m[0m [0m[1m[33mWarning: [0m[0m[1mNo outputs found[0m
[33m[0m [0m
[33m[0m [0mThe state file either has no outputs defined, or all the defined outputs
[33m[0m [0mare empty. Please define an output in your configuration with the `output`
[33m[0m [0mkeyword and run `terraform refresh` for it to become available. If you are
[33m[0m [0musing interpolation, please verify the interpolated value is not empty. You
[33m[0m [0mcan use the `terraform console` command to assist.
[33m[0m[0m
+ ls -l sunitChintu2.txt
-rwxr-xr-x. 1 jenkins jenkins 31 Oct 23 10:35 sunitChintu2.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
[ec2-user@ip-172-31-19-186 workspace]$
[ec2-user@ip-172-31-19-186 workspace]$
[ec2-user@ip-172-31-19-186 workspace]$ pwd
/var/lib/jenkins/workspace
[ec2-user@ip-172-31-19-186 workspace]$ cd terraform-pipeline-lab
[ec2-user@ip-172-31-19-186 terraform-pipeline-lab]$ ll
total 12
-rw-r--r--. 1 jenkins jenkins  358 Oct 23 10:35 main.tf
-rwxr-xr-x. 1 jenkins jenkins   31 Oct 23 10:35 sunitChintu2.txt
-rw-r--r--. 1 jenkins jenkins 2157 Oct 23 10:35 terraform.tfstate
[ec2-user@ip-172-31-19-186 terraform-pipeline-lab]$
```



4. Conclusion

Terraform was successfully integrated into a Jenkins Declarative Pipeline. The Pipeline initialized the required providers, generated a plan, applied the Terraform configuration, and created sunitChintu2.txt with the expected content.

This lab demonstrated how Jenkins Pipeline automates Terraform execution and verifies the result without creating AWS infrastructure.

Task3: Create one Jenkins job using Maven Project for the code below with two stages:

- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation **Code:** https://github.com/betawins/java-Working-app.git

1. Objective

To configure a Jenkins Maven Project job that retrieves source code from GitHub, executes Maven build goals, and generates a WAR artifact.

2. Purpose

This practical demonstrates how Jenkins integrates with Git and Maven to automate the application build process.

Jenkins retrieves the project from a GitHub repository, reads the Maven configuration from pom.xml, executes the configured Maven goals, and generates the build output in the Jenkins workspace.

This is a fundamental Continuous Integration (CI) workflow used in DevOps.

3. Environment and Prerequisites

The following environment was used for this practical:

- **Jenkins:** 2.580.1
- **Operating system:** Amazon Linux 2023
- **Linux user:** ec2-user
- **Maven:** 3.8.4
- **Java:** Java 17 was selected by Maven in the earlier environment check; Java 21 is also installed.
- **Git repository:** https://github.com/betawins/java-Working-app.git
- **Branch:** main
- **Jenkins job name:** Java-Working-App-Maven

The required tools were installed, and Jenkins was running and accessible through its web interface.

4. Jenkins Job Configuration

Step 1: Create a new Jenkins job

1. Log in to the Jenkins dashboard.
2. Click **New Item**.
3. Enter the job name:

Java-Working-App-Maven

4. Select **Maven project** as the job type.
5. Click **OK** to open the configuration page.

Step 2: Configure GitHub Source Code Management

Under the **Source Code Management** section:

- Select **Git**.
- Enter the repository URL.
- Leave credentials as - none - because the repository is public.
- Specify the branch to build.

Configuration:
Repository URL: https://github.com/betawins/java-Working-app.git
Credentials: - none -
Branch Specifier: */main

Explanation:

- The repository URL tells Jenkins where to retrieve the project.
- Credentials are unnecessary for cloning this public repository.
- */main instructs Jenkins to build the main branch.

The saved job configuration was inspected on the EC2 instance, and the configured Git URL and branch specifier were confirmed.

Step 3: Configure the Maven build

Under the **Build** section, configure the following values:

Root POM: pom.xml

Maven Goals and options: clean compile

Step 4: Save the configuration

1. Review the Git and Maven settings.
2. Click **Save**.
3. Open the Java-Working-App-Maven job page.

The job is now configured to check out the repository and execute the specified Maven goals.

5. Execute the Jenkins Build

Step 1: Start the build

1. Open Java-Working-App-Maven.
2. Click **Build Now**.
3. Wait for the build to finish.
4. Open the build number.
5. Click **Console Output**.

The Console Output displays the Git checkout process, Maven execution, warnings, and final build result.

6. Verify the Jenkins Workspace

After the build, the workspace was inspected directly on the EC2 instance.

Step 1: Navigate to the Jenkins workspace

cd /var/lib/jenkins/workspace/Java-Working-App-Maven

7. Verify the Generated WAR Artifact

Step 1: Navigate to the target directory

cd /var/lib/jenkins/workspace/Java-Working-App-Maven/target

Step 2: List the generated files

ll

The observed output was:

drwxr-xr-x. 4 jenkins jenkins 54 Oct 26 11:39 hiring

-rw-r--r--. 1 jenkins jenkins 1698 Oct 26 11:39 hiring.war

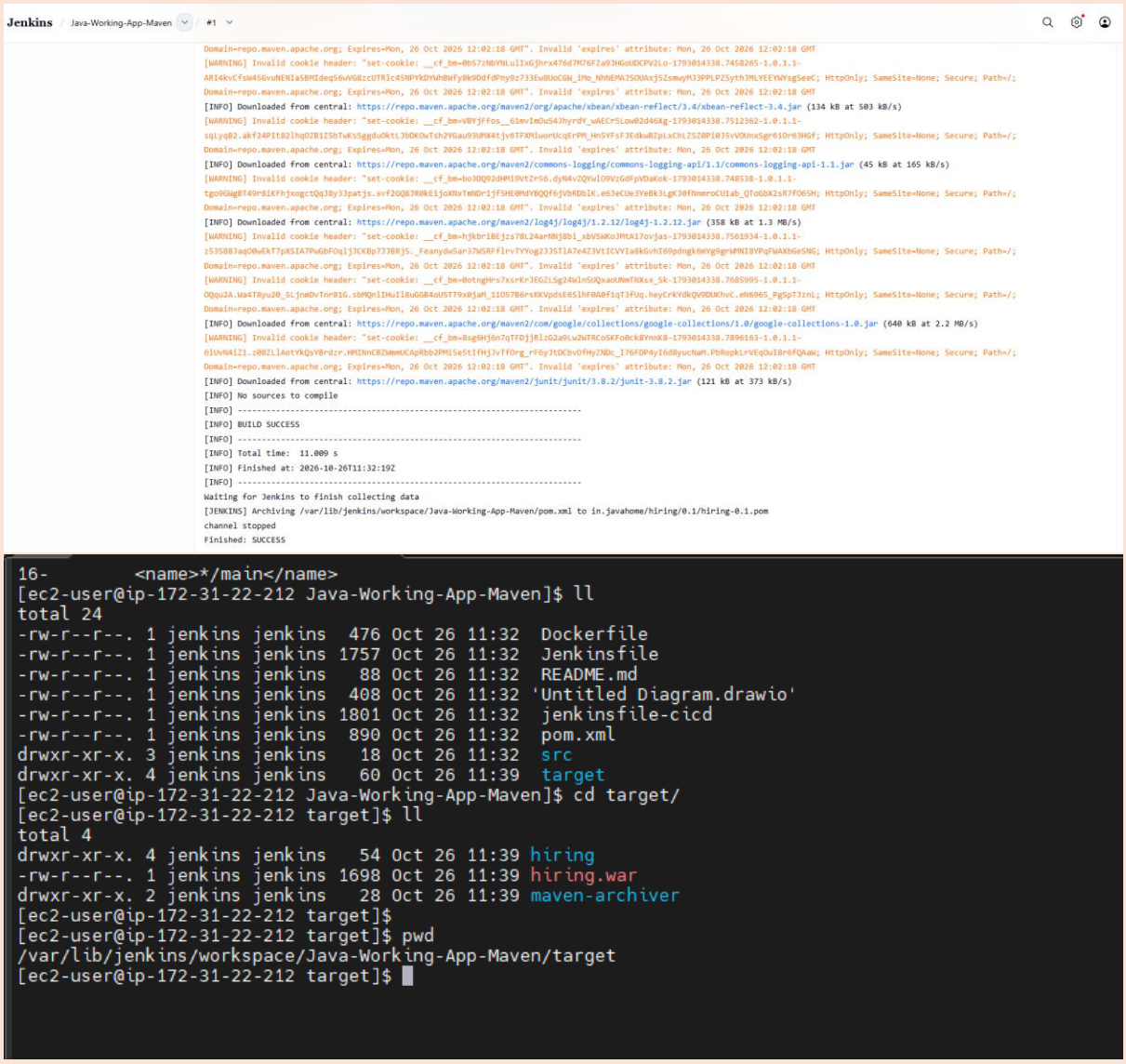
drwxr-xr-x. 2 jenkins jenkins 28 Oct 26 11:39 maven-archiver

Step 3: Verify the WAR file directly

ls -lh /var/lib/jenkins/workspace/Java-Working-App-Maven/target/hiring.war

The WAR artifact was present, with an observed size of **1,698 bytes**.

8.Implementation:



9. Conclusion

The Java-Working-App-Maven Jenkins job was configured to retrieve the GitHub repository's main branch and execute Maven using pom.xml and the clean compile goals.

Jenkins reported Finished: SUCCESS, Maven reported BUILD SUCCESS, and the workspace contained the generated artifact target/hiring.war.

Therefore, the **Maven build and WAR packaging were successful** in this run.

However, the build log reported No sources to compile, and no .java files were found in the checked-out source tree. Consequently, actual Java source compilation has not been demonstrated. If Java compilation is an explicit requirement of the assignment, the intended repository contents or source location must be confirmed before marking that part complete.

Task 4:Use the same code and create a parameterized job in Jenkins with:

- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation **Code:** https://github.com/betawins/java-Working-app.git

1. Objective

To create a parameterized Jenkins job that checks out the application from GitHub and executes Maven using a user-selected build option.

2. Purpose

Parameterized jobs allow users to provide values at build time instead of hardcoding every option in the job configuration.

For this practical, we will create a choice parameter named MAVEN_GOALS with two options:

- clean compile — cleans previous build output and runs the compilation phase.
- clean package — cleans previous build output and runs the package phase, which can generate the WAR artifact.

We will reuse the same GitHub repository and Maven POM from Task 2.

3. Step-by-Step Implementation

Step 1: Create a new Jenkins job

1. Open the Jenkins dashboard.
2. Click New Item.
3. Enter this job name: Java-Working-App-Maven-Parameterized
4. Select Maven project.
5. Click OK.

Keep the original Java-Working-App-Maven job unchanged.

Step 2: Enable parameterization

On the new job's configuration page:

1. Find This project is parameterised.
2. Check the box.
3. Click Add Parameter.
4. Select Choice Parameter.

Configure it as follows:
Choice Parameter configuration

Name:
MAVEN_GOALS

Choices

clean compile
clean package

Description: Select the Maven goals to execute for this build.

The two choices should be entered on separate lines.

Step 3: Configure GitHub

Under Source Code Management, select Git and use the same settings as Task 2.

Repository URL: https://github.com/betawins/java-Working-app.git

Credentials: - none -

Branch Specifier: */main

Because the repository is public, credentials are not required for this checkout.

Step 4: Configure the Maven goals

Under the Build section, configure:

Root POM: pom.xml

Maven Goals and options: $MAVEN_GOALS

The $MAVEN_GOALS value is intended to be replaced by the selected Jenkins parameter when the build runs.

For example:

- If you select clean compile, Jenkins should run those goals.
- If you select clean package, Jenkins should run those goals and attempt to create the WAR artifact.

Leave the Terraform Environment option unchecked. It is not needed for this task.

Step 5: Save the job

Click Save.

Your new job should now be available on the Jenkins dashboard:

Java-Working-App-Maven-Parameterized

4. Execute the Parameterized Build

Test 1: Run with clean compile

1. Open the new job.
2. Click Build with Parameters.
3. Select clean compile for MAVEN_GOALS.
4. Click Build.
5. Open the build number and select Console Output.

Check that Maven executes the selected goals and that the build finishes successfully.

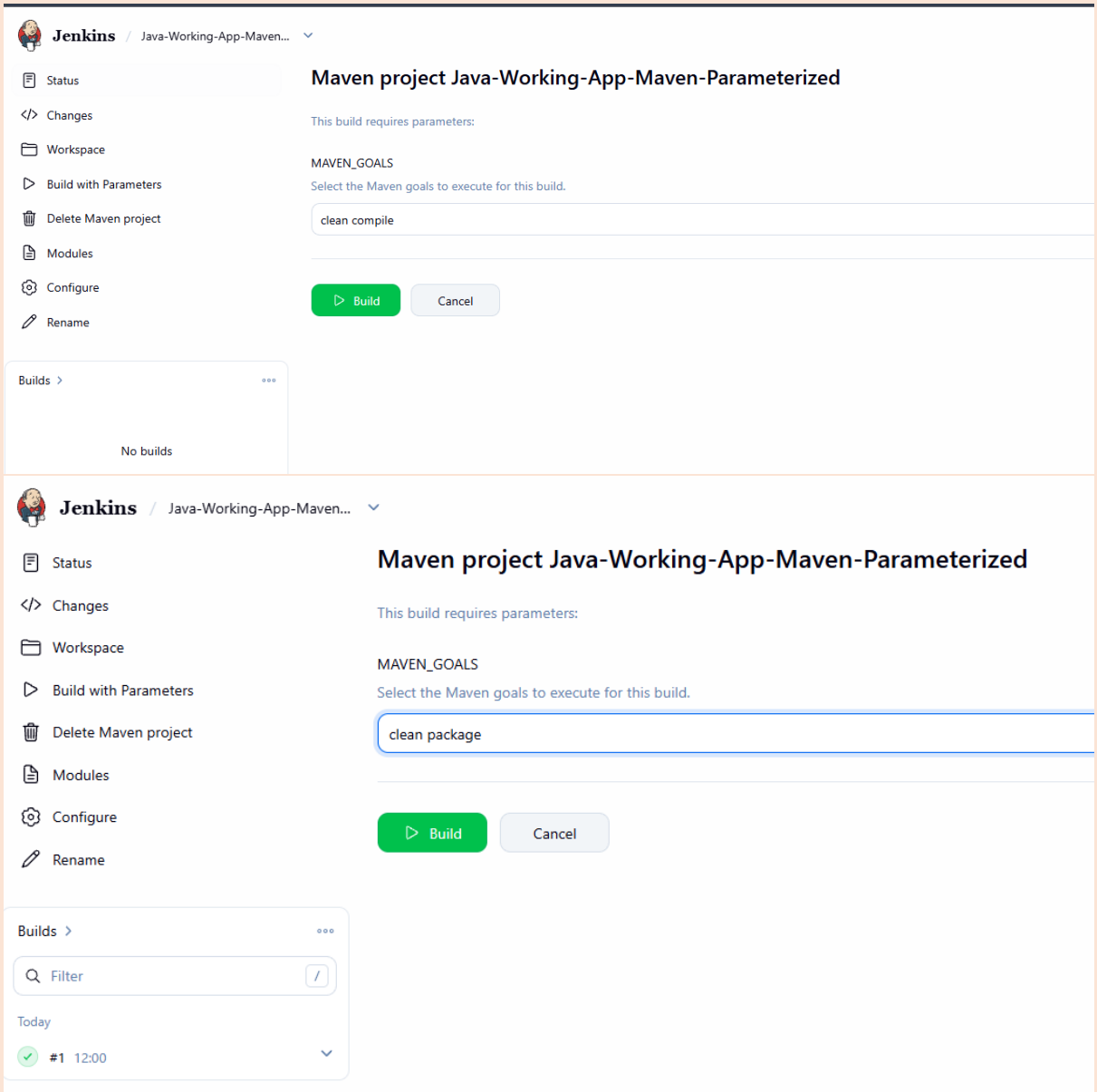
Test 2: Run with clean package

1. Return to the job.
2. Click Build with Parameters again.

3. Select clean package.
4. Click Build.
5. Open the new build's Console Output.

If successful, verify the WAR file:

5. Implementation:






```
[ec2-user@ip-172-31-22-212 target]$ pwd
/var/lib/jenkins/workspace/Java-Working-App-Maven/target
[ec2-user@ip-172-31-22-212 target]$
[ec2-user@ip-172-31-22-212 target]$ cd ..
[ec2-user@ip-172-31-22-212 Java-Working-App-Maven]$ cd ..
[ec2-user@ip-172-31-22-212 workspace]$ ll
total 0
drwxr-xr-x. 5 jenkins jenkins 185 Oct 26 11:39 Java-Working-App-Maven
drwxr-xr-x. 5 jenkins jenkins 185 Oct 26 12:01 Java-Working-App-Maven-Parameterized
drwxr-xr-x. 3 jenkins jenkins  29 Oct 26 11:16 Terraform-Integration-Check
[ec2-user@ip-172-31-22-212 workspace]$ cd Java-Working-App-Maven-Parameterized/
[ec2-user@ip-172-31-22-212 Java-Working-App-Maven-Parameterized]$ ll
total 24
-rw-r--r--. 1 jenkins jenkins  476 Oct 26 12:00  Dockerfile
-rw-r--r--. 1 jenkins jenkins 1757 Oct 26 12:00  Jenkinsfile
-rw-r--r--. 1 jenkins jenkins   88 Oct 26 12:00  README.md
-rw-r--r--. 1 jenkins jenkins  408 Oct 26 12:00 'Untitled Diagram.drawio'
-rw-r--r--. 1 jenkins jenkins 1801 Oct 26 12:00  jenkinsfile-cicd
-rw-r--r--. 1 jenkins jenkins  890 Oct 26 12:00  pom.xml
drwxr-xr-x. 3 jenkins jenkins   18 Oct 26 12:00  src
drwxr-xr-x. 4 jenkins jenkins   60 Oct 26 12:01  target
[ec2-user@ip-172-31-22-212 Java-Working-App-Maven-Parameterized]$ cd target/
[ec2-user@ip-172-31-22-212 target]$ ll
total 4
drwxr-xr-x. 4 jenkins jenkins   54 Oct 26 12:01 hiring
-rw-r--r--. 1 jenkins jenkins 1698 Oct 26 12:01 hiring.war
drwxr-xr-x. 2 jenkins jenkins   28 Oct 26 12:01 maven-archiver
[ec2-user@ip-172-31-22-212 target]$
```

6. Conclusion

This practical creates a separate parameterized Jenkins Maven Project using the same GitHub repository and main branch as Task 2. The user can select Maven goals at build time, allowing the job to execute either clean compile or clean package.

Once both selections have been tested and their results verified, Task 3 will be complete.

Your next action: Create the job and run the first test with clean compile. Share the Console Output or the exact error if it fails, and we'll verify the parameter substitution before moving to the final task on Jenkins global variables.

Task 5:What are the global variables in Jenkins?

1. Objective

To understand Jenkins environment variables, identify the commonly used variables, and demonstrate how to display them during a Jenkins build.

2. Purpose

Jenkins provides environment variables that describe the current job, build number, workspace, Jenkins server, and source-control checkout.

These variables help DevOps engineers automate builds, create artifact names, locate files, troubleshoot pipelines, and pass build information between steps without hardcoding values.

3. Important Jenkins Environment Variables

Variable	Meaning	Example or use
BUILD_NUMBER	Current build number	5
BUILD_ID	Identifier for the current build	Build identification
BUILD_TAG	Unique build-related tag	Labeling artifacts
BUILD_URL	URL of the current build, when configured	Build links in notifications
JOB_NAME	Full job name	Java-Working-App-Maven
JOB_BASE_NAME	Job name without folder path	Java-Working-App-Maven
JOB_URL	URL of the job, when configured	Link to the job
WORKSPACE	Directory assigned to the job	/var/lib/jenkins/workspace/...
JENKINS_HOME	Jenkins controller data directory	/var/lib/jenkins
JENKINS_URL	Jenkins root URL, when configured	Used in links and notifications
NODE_NAME	Name of the node executing the build	Controller or agent name
NODE_LABELS	Labels assigned to that node	Agent selection
EXECUTOR_NUMBER	Executor slot number on the node	Identifies the executor
GIT_BRANCH	Branch information supplied by the Git plugin	Git checkout diagnostics
GIT_COMMIT	Checked-out Git commit hash, when supplied by the Git plugin	Traceability

Important: Availability and values depend on the job type, installed plugins, node, and Jenkins configuration. For example, Git-related variables are not guaranteed to be available in every kind of job.

4. Practical: Display Environment Variables in Jenkins

Step 1: Create a Freestyle job

1. Open the Jenkins dashboard.
2. Click New Item.
3. Enter the job name: Jenkins-Global-Variables
4. Select Freestyle project.
5. Click OK.

Step 2: Add an Execute shell build step

Under Build Steps:

1. Click Add build step.
2. Select Execute shell.
3. Add the following commands.

```
#!/bin/bash

echo "===== Jenkins Global Variables ====="

echo "Job Name: $JOB_NAME"
echo "Build Number: $BUILD_NUMBER"
echo "Build ID: $BUILD_ID"
echo "Build Tag: $BUILD_TAG"
echo "Workspace: $WORKSPACE"
echo "Jenkins Home: $JENKINS_HOME"
echo "Node Name: $NODE_NAME"
echo "Node Labels: $NODE_LABELS"
echo "Executor Number: $EXECUTOR_NUMBER"
echo "Build URL: $BUILD_URL"
echo "Job URL: $JOB_URL"
echo "Jenkins URL: $JENKINS_URL"

echo "===== Git Variables (if available) ====="
echo "Git Branch: ${GIT_BRANCH:-Not available}"
echo "Git Commit: ${GIT_COMMIT:-Not available}"

echo "===== All Environment Variables ====="
printenv | sort
```

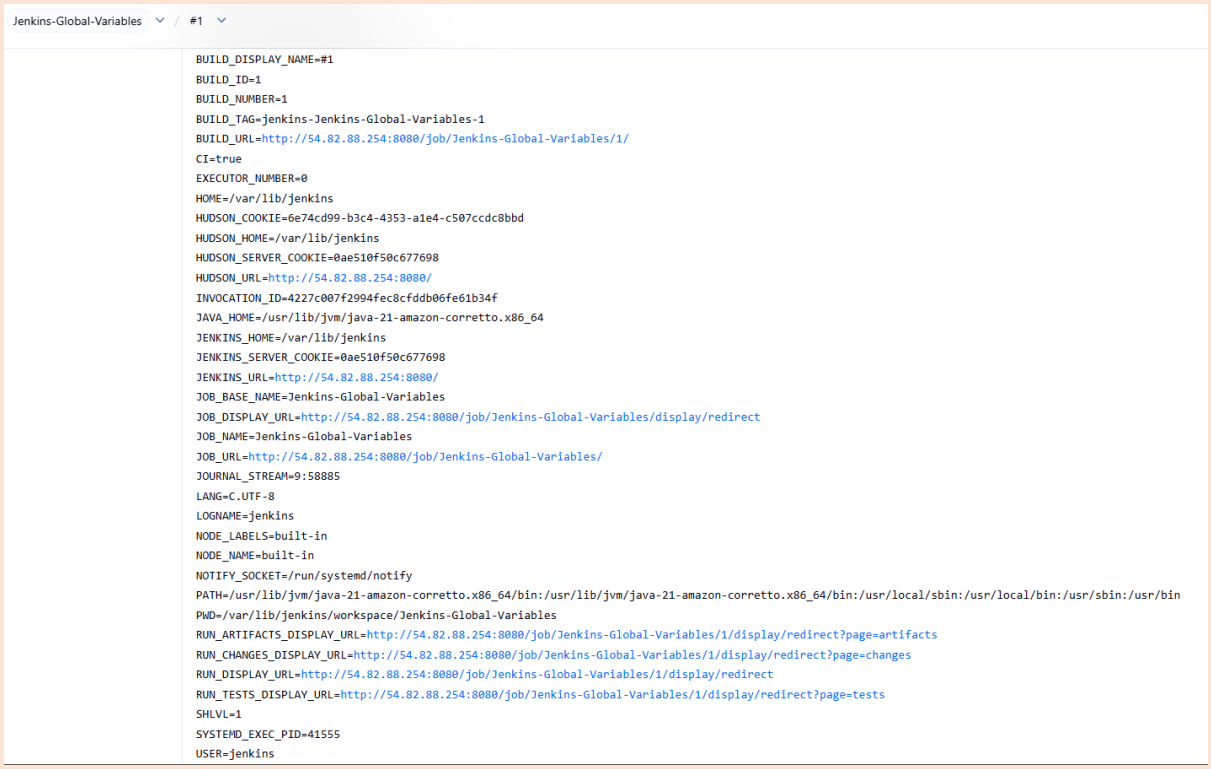
The ${VARIABLE:-Not available} syntax prevents the script from printing a confusing blank value when a Git variable is unavailable.

Step 3: Save and run the job

1. Click Save.
2. Open Jenkins-Global-Variables.
3. Click Build Now.
4. Open the build number.
5. Click Console Output.

You should see the values Jenkins supplied for the current build, followed by the available environment variables.

6. Implementation



8. Conclusion

This practical demonstrates how to create a Jenkins Freestyle job and inspect environment variables using an Execute shell step. These variables make builds more dynamic, improve traceability, reduce hardcoded paths, and support automation across CI/CD workflows.

With this task, the planned Jenkins assignment topics are covered:

1. Terraform integration in Jenkins.
2. Maven Project job using GitHub.
3. Parameterized Maven job.
4. Jenkins global environment variables.

Next: Run the global-variable job and save its Console Output as evidence. If your course requires a submission document, we can consolidate all four tasks into one consistent document using your preferred Objective → Purpose → Steps → Verification → Conclusion format.